

INTRODUCTION TO RISC-V

# RISC-V入门教程

ISA | 汇编指令 | 系统编程 | 组成原理 | 嵌入式应用

## 从零开始实现RV32I模拟器

主讲 邢建国



中国开放指令生态 (RISC-V) 联盟 | 浙江中心  
China RISC-V Alliance | Zhejiang Center



浙江图灵算力研究院  
ZHEJIANG TURING INSTITUTE

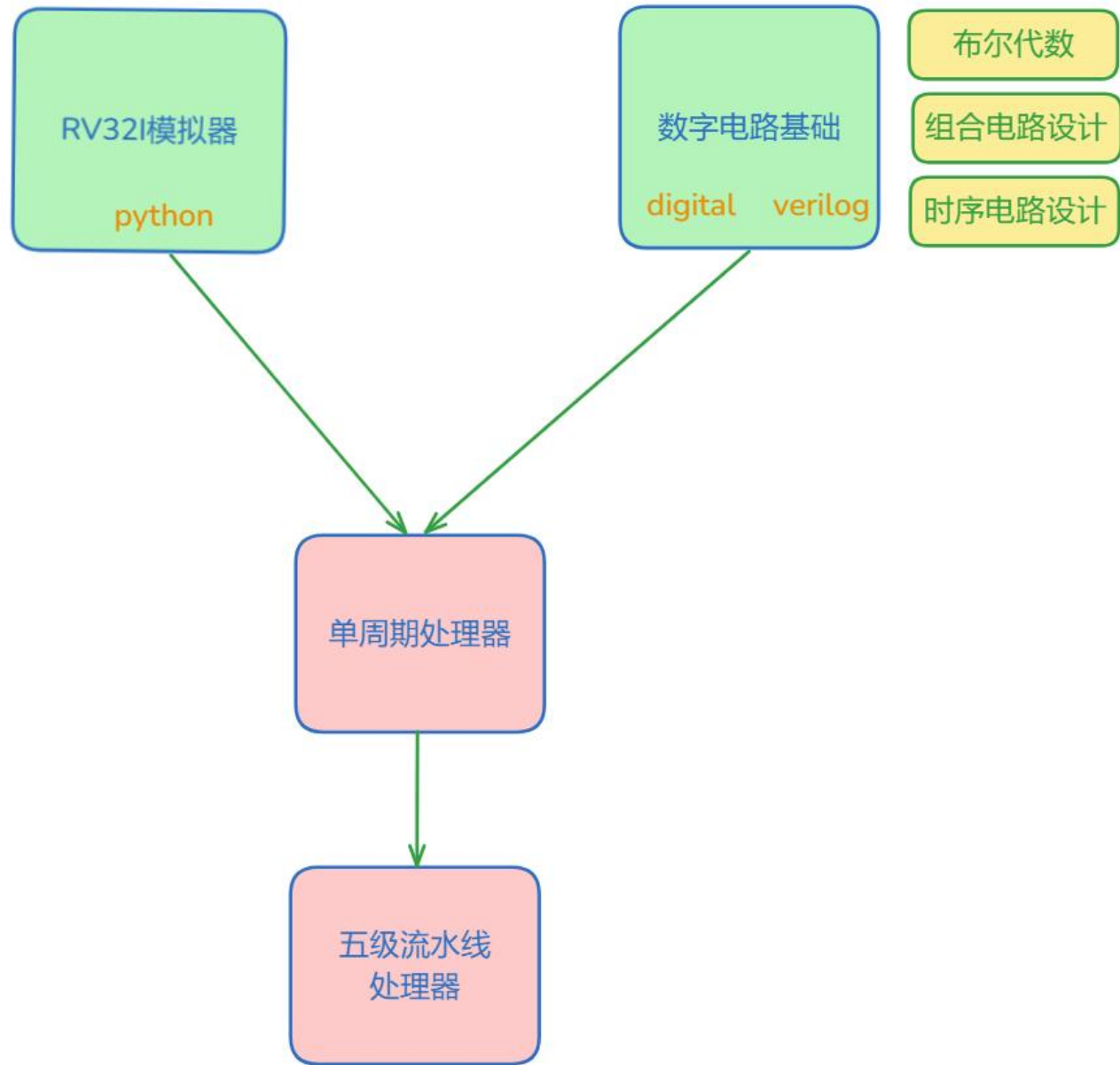




## RV32I模拟器



- 模拟器结构
- LUI、AUIPC指令
- JAL、JALR、Branch指令
- 算术逻辑运算指令
- Load、store指令及外设模拟
- CSR及特权指令



## ■ 模拟器

模拟器用于模拟RISC-V指令集架构（ISA）的行为，允许开发者在没有实际硬件的情况下测试和运行RISC-V程序。

- 模拟RISC-V ISA：RISC-V模拟器专注于模拟RISC-V指令集，包括基础的V32I指令集以及各种扩展，如RV64I、RV32M等。
- 教育与研究：模拟器是学习和研究RISC-V架构的理想工具，可以通过模拟器深入理解RISC-V指令集及其扩展。
- 软件开发与测试：开发者可以在模拟器上编写和测试RISC-V架构的软件，确保代码在实际硬件上的正确性和性能。
- 硬件验证：硬件设计者可以利用模拟器进行功能验证，确保硬件实现与ISA规范的一致性。

## ■ 模拟器分类

CPU模拟器可以根据不同的模拟层次和用途被分为几个主要类别：

- **功能级模拟器** (Behavioral Simulators) :

功能级模拟器主要关注于模拟CPU的行为和功能，而不涉及具体的时钟周期或硬件细节。它们通常用于早期的设计验证和软件开发，因为它们能够快速模拟处理器的行为。

- **时钟级模拟器** (Cycle-Accurate Simulators) :

时钟级模拟器模拟了CPU的微架构和时钟周期，提供了更精确的模拟，包括指令的执行时间、流水线的行为等。这类模拟器通常用于性能分析和架构探索。

- **全系统模拟器** (Full-System Simulators) :

全系统模拟器不仅模拟CPU，还包括内存、I/O设备和操作系统等，用于模拟整个计算机系统的行为。它们可以运行完整的操作系统和应用程序，适用于软件开发和系统级测试。

- **微架构探索和性能优化模拟器** :

这类模拟器专注于精准模拟处理器微结构，以进行性能优化和微架构分析。它们可以进一步细分为单核性能模拟器和多核互联性能模拟器。

## ■ 已有的RISC-V模拟器

- QEMU模拟器：QEMU是一个广泛使用的开源系统模拟器，支持多种架构，包括RISC-V。通过QEMU，可以在x86笔记本或其他平台上模拟RISC-V硬件环境，并运行RISC-V操作系统和应用程序。QEMU支持多种模式，包括用户模式和系统模式，适用于不同的开发需求。
- <https://risc-v-getting-started-guide.readthedocs.io/en/latest/linux-qemu.html>
- Spike模拟器：Spike是RISC-V基金会推荐的一个高效、灵活的模拟器，支持RV32I指令集，并具备丰富的调试功能，如断点设置和单步执行。Spike适用于学术研究、工业生产以及嵌入式系统设计，帮助开发者验证设计并减少硬件调试时间。
- <https://github.com/riscv-software-src/riscv-isa-sim>

## ■ 已有的模拟器

riscv-em模拟器：这是一个由C语言编写的开源RISC-V ISA模拟器，支持RV32IMA和RV64IMA指令集，并集成了CLINT、PLIC、UART等关键组件。riscv-em主要用于教育和测试软件兼容性，适合初学者和工程师使用。

pyriscv模拟器：这是一个基于Python开发的RISC-V数据精确模拟器，支持RV32I指令集，代码量较少且易于理解和维护。pyriscv适合快速原型开发和教学用途。

rv8模拟器：rv8是一个高性能的RISC-V模拟套件，支持x86-64二进制翻译和全系统模拟。它适用于需要高性能和复杂功能的场景，如ELF二进制分析和ISA元数据生成。

其他工具和平台：除了上述模拟器外，还有许多其他工具和平台支持RISC-V的开发和测试，如SweRV ISS、**RARS**、OpenOCD等。

## ■ 为什么要写一个RV32I模拟器

- 已有RISC-V模拟器过于复杂，不是学习计算机体系结构和CPU设计的理想平台。
- 通过编写模拟器，可以直观地理解每条指令的执行流程，为掌握从指令集到硬件实现的提供基础。
- 在实际硬件实现之前，模拟器可以用于进行功能性和性能测试，减少后期硬件调试的时间成本。
- 可以了解交叉编译、内存镜像、外设等细节。

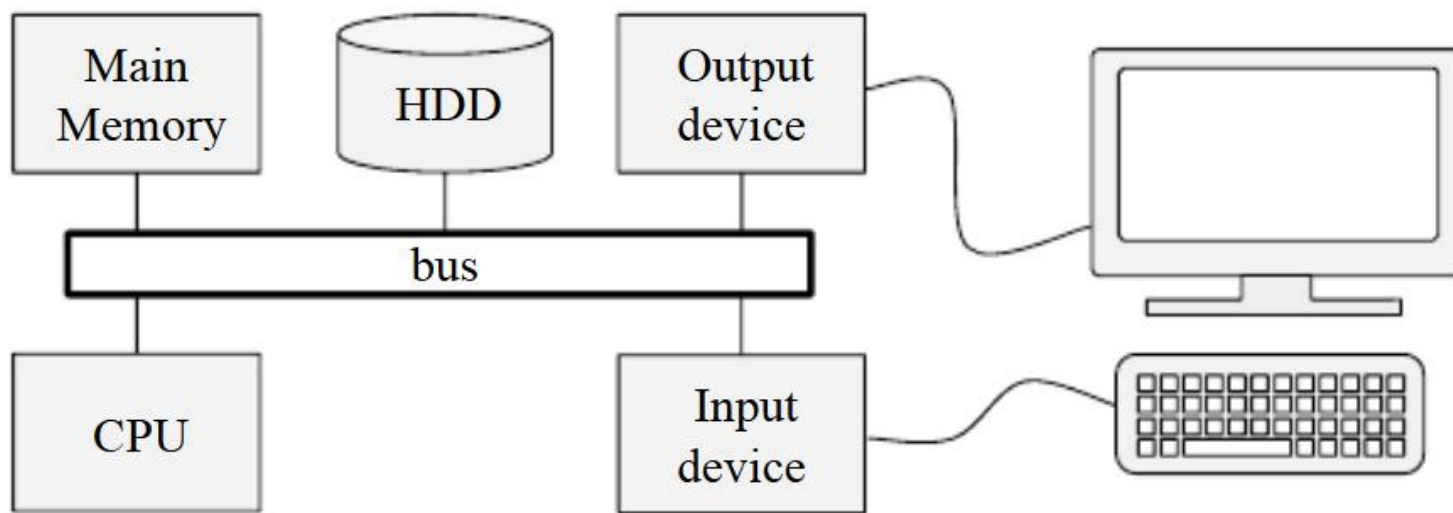


## ■ 为什么使用Python

- 易学易用：Python是一种高级编程语言，以其简洁明了的语法而闻名，易于学习和使用。这使得初学者和专家都能够快速上手编写模拟器。
- 快速开发：Python的开发效率高，可以快速实现模拟器原型，进行测试和迭代，这对于教育和研究尤其有价值。
- 跨平台：Python是跨平台的，可以在多种操作系统上运行，如Windows、Linux和macOS，这使得Python编写的模拟器具有很好的可移植性。
- 只使用了Python的基本数据结构：list、dict，很容易用其他语言如C、C++、java、golang等重写。

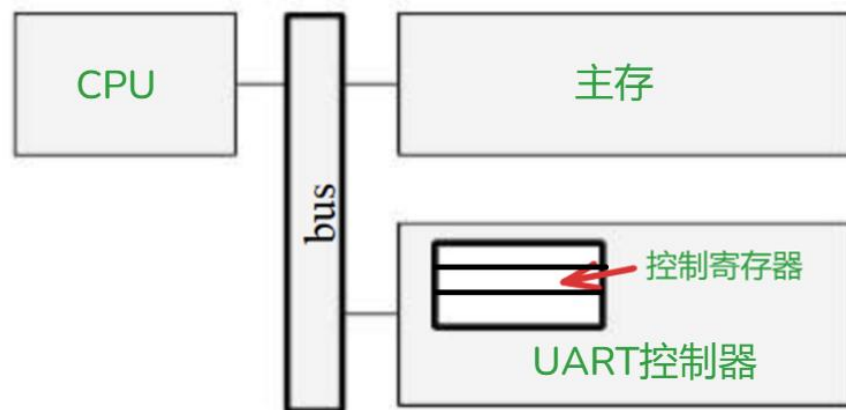
## ■ 模拟器结构

计算机系统由CPU、主存和输入输出设备（外设）组成，各部分之间通过总线连接。



## ■ 模拟器结构

- RV32I zicsr处理器
  - 基本指令+特权指令
  - 机器模式和用户模式
- 内存
  - 指令和数据
- 串口 (UART 16650)
  - 输入
  - 输出



地址: 0x8000\_0000 - 0xffff\_ffff

地址: 0x1000\_0000 - 0x1000\_0010

## ■ 内存

- 起始地址: 0x8000\_0000
- 大小: 由运行程序镜像大小决定

- 数据结构

```
mem = {  
    'mem': None,  
    'base': 0,  
    'size': 0,  
}
```

- 接口函数

- 读内存

```
mem_load(mem, addr, size)
```

- 写内存

```
mem_store(mem, addr, size, val)
```

- 检测地址范围

```
check_addr(mem, addr)
```

- 加载程序镜像

```
load_image(mem, file, base = mem_base)
```

## ■ 内存

- 读内存

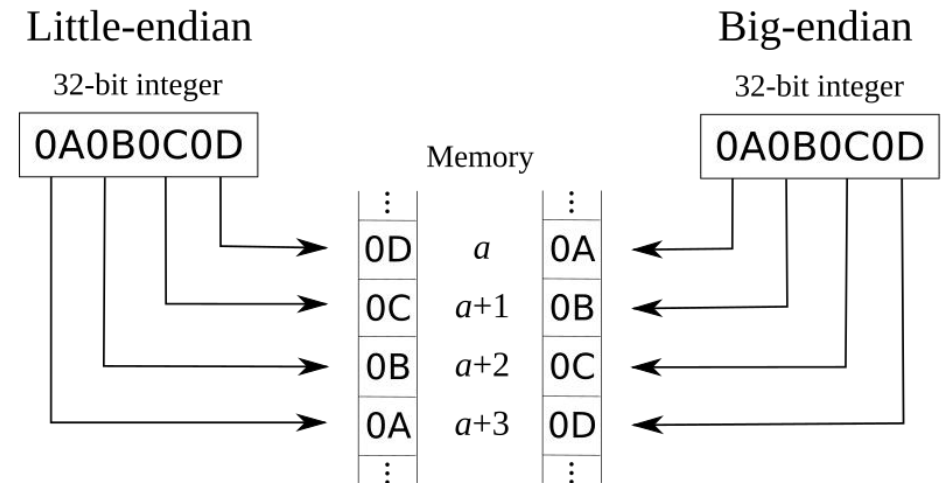
```
def mem_load(mem, addr, size):  
    addr = addr - mem['base']  
    return int.from_bytes(bytes(mem['mem'][addr:addr + size]), 'little')
```

小端方式存储



- 写内存

```
def mem_store(mem, addr, size, val):  
    addr = addr - mem['base']  
  
    for i in range(size):  
        mem['mem'][i + addr] = val & 0xff  
        val = val >> 8
```



## ■ 内存

- 加载程序镜像

```
def load_image(mem, file, base = mem_base):  
    f = open(file, 'rb')  
    size = os.path.getsize(file)  
  
    buf = f.read(size)  
  
    f.close()  
  
    mem['mem'] = [i for i in buf]  
  
    mem['base'] = base  
    mem['size'] = size
```

- 地址检查

```
def check_addr(mem, addr):  
    print(addr, addr - mem['base'], mem['size'])  
  
    if addr < 0 or (addr - mem['base']) >= mem['size'] :  
        return False  
    else:  
        return True
```

## ■ 如何得到程序镜像

```
.global _start

.data
x: .word 100
y: .word 200
z: .word 0
stack_end: .skip 1024
stack_start:

.text
_start:

    la sp, stack_start

    lw a0, x
    lw a1, y
    add a2, a1, a0

    call putch

    sw a2, z, t0

    j start
```

?



```
00000000 17 11 00 00 13 01 81 4b 17 15 00 00 03 25 45 0a
00000020 97 15 00 00 83 a5 05 0a 33 86 a5 00 97 00 00 00
00000040 e7 80 80 04 97 12 00 00 23 a8 c2 08 6f f0 5f fd
00000060 97 00 00 00 e7 80 80 05 97 00 00 00 e7 80 c0 02
00000100 6f f0 1f ff 13 06 05 00 13 05 00 00 93 05 00 00
00000120 63 48 b6 00 33 85 a5 00 93 85 15 00 6f f0 5f ff
00000140 67 80 00 00 13 01 01 ff 23 20 11 00 23 22 81 00
00000160 37 04 00 10 23 00 a4 00 83 20 01 00 03 24 41 00
00000200 13 01 01 01 67 80 00 00 13 01 01 ff 23 20 11 00
00000220 23 22 81 00 37 04 00 10 03 05 04 00 83 20 01 00
00000240 03 24 41 00 13 01 01 01 67 80 00 00 00 00 00 00
00000260 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
*
0010240 00 00 00 00 00 00 00 00 00 00 00 00 00 64 00 00 00
0010260 c8 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0010300 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
*
0012260 00 00 00 00 00 00 00 00 00
0012270
```



## ■ 镜像制作

- RISC-V交叉编译环境搭建
- xPack GNU GCC:  
<https://github.com/xpack-dev-tools/riscv-none-elf-gcc-xpack/releases>
- OSX
- Linux
- Windows
- 在windows下解压  
path\to\install
- 然后添加到系统路径

### ▼ Assets

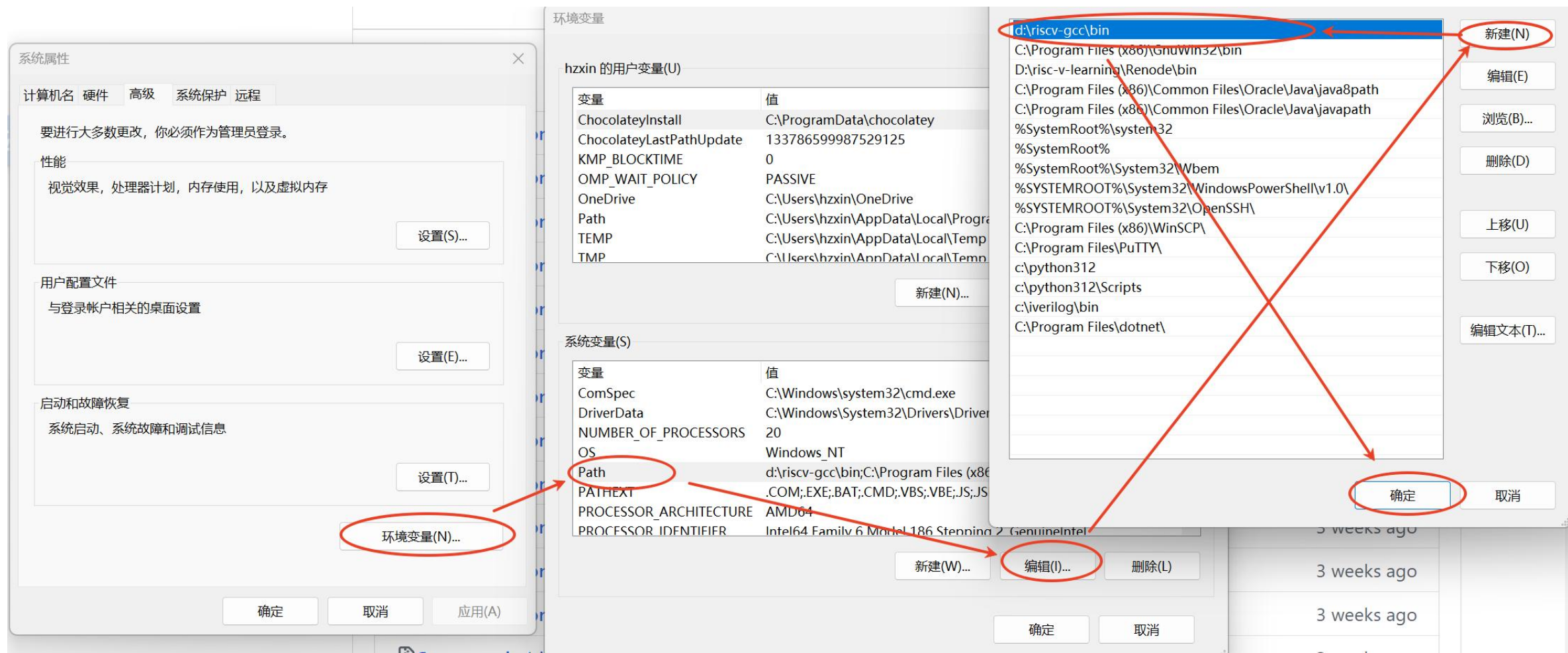
14

|                                                                                                                                               |           |
|-----------------------------------------------------------------------------------------------------------------------------------------------|-----------|
|  xpack-riscv-none-elf-gcc-14.2.0-3-darwin-arm64.tar.gz     | 316 MB    |
|  xpack-riscv-none-elf-gcc-14.2.0-3-darwin-arm64.tar.gz.sha | 120 Bytes |
|  xpack-riscv-none-elf-gcc-14.2.0-3-darwin-x64.tar.gz       | 319 MB    |
|  xpack-riscv-none-elf-gcc-14.2.0-3-darwin-x64.tar.gz.sha   | 118 Bytes |
|  xpack-riscv-none-elf-gcc-14.2.0-3-linux-arm.tar.gz        | 315 MB    |
|  xpack-riscv-none-elf-gcc-14.2.0-3-linux-arm.tar.gz.sha    | 117 Bytes |
|  xpack-riscv-none-elf-gcc-14.2.0-3-linux-arm64.tar.gz      | 324 MB    |
|  xpack-riscv-none-elf-gcc-14.2.0-3-linux-arm64.tar.gz.sha  | 119 Bytes |
|  xpack-riscv-none-elf-gcc-14.2.0-3-linux-x64.tar.gz        | 328 MB    |
|  xpack-riscv-none-elf-gcc-14.2.0-3-linux-x64.tar.gz.sha    | 117 Bytes |
|  xpack-riscv-none-elf-gcc-14.2.0-3-win32-x64.zip           | 361 MB    |
|  xpack-riscv-none-elf-gcc-14.2.0-3-win32-x64.zip.sha     | 114 Bytes |
|  Source code (zip)                                       |           |
|  Source code (tar.gz)                                    |           |

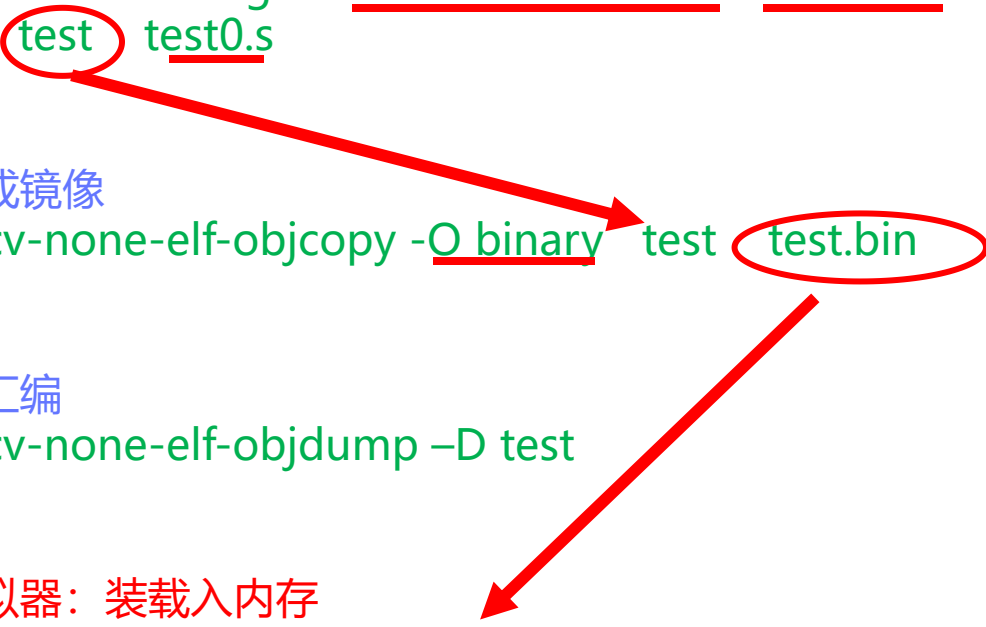


## ■ Windows下交叉编译环境安装

- 在windows下解压至目录，如：d:\riscv-gcc；添加d:\riscv-gcc\bin到系统环境变量path



## ■ 镜像制作

- 编译生成elf格式的可执行文件
    - `riscv-none-elf-gcc -Ttext=0x8000000 -mno-relax -march=rv32i_zicsr -nostdlib -ffreestanding`
    - `-o test test0.s`
  - 生成镜像
    - `riscv-none-elf-objcopy -O binary test test.bin`
  - 反汇编
    - `riscv-none-elf-objdump -D test`
  - 模拟器：装载入内存
    - `load_image(mem, 'test.bin', 0x8000_0000)`
- 

## ■ CPU

CPU包括:

- pc: 程序寄存器
  - 程序入口地址 (0x8000\_0000)
- regs: 32个32位整数寄存器
- csr: 1024个32位控制状态寄存器
- 其他信息:
  - trap: 异常或中断标志
  - trap\_val

- 指令执行或外设中断都可能改变cpu中pc、regs、csr以及trap、trap\_val的内容

```
cpu = {  
    'pc': 0,  
    'regs': [0 for _ in range(32)],  
    'csr': [0 for _ in range(1024)],  
    'trap': 0,  
    'trap_val': 0,  
}
```

## ■ CPU接口函数

- 取指令 `fetch(cpu, mem)`
- 执行指令 `execute(cpu, mem, inst)`
- 单步执行 `step(cpu, mem)`
- 辅助函数
  - `get_pc`、`set_pc`、`pc_inc4`
  - `get_reg`、`set_reg`
  - `get_csr`、`set_scr`

## ■ CPU接口函数

- 取指令

```
def fetch(cpu, mem):  
    pc = get_pc(cpu)  
    return mem_load(mem, pc, 4)
```

- 执行指令

```
def execute(cpu, mem, inst):  
    #todo  
    pc_inc4(cpu)
```

- 单步执行

```
def step(cpu, mem):  
    inst = fetch(cpu, mem)  
  
    #显示指令地址，指令的16进制和二进制编码  
    log(f'{hex(cpu["pc"]):<{10}}: {hex(inst):<{10}} ---- {bin(inst):<{36}}')  
  
    execute(cpu, mem, inst)
```

## ■ 运行模拟器

```
def run(cpu, mem, image_file, image_base=0x8000_0000, start=0x8000_0000):  
    load_image(mem, image_file, image_base)  
    set_pc(cpu, start)
```

```
    while True:  
        step(cpu, mem)
```

```
run(cpu, mem, 'test.bin')
```



```
>>> %Run rv32i.py
```

```
0x80000000: 0x1117 ---- 0b1000100010111  
0x80000004: 0x4b810113 ---- 0b1001011100000010000000100010011  
0x80000008: 0x1517 ---- 0b1010100010111  
0x8000000c: 0xa452503 ---- 0b101001000101001001010000001  
0x80000010: 0x1597 ---- 0b1010110010111  
0x80000014: 0xa05a583 ---- 0b101000000101101001011000001  
0x80000018: 0xa58633 ---- 0b101001011000011000110011  
0x8000001c: 0x97 ---- 0b10010111  
0x80000020: 0x48080e7 ---- 0b10010000001000000011100111  
0x80000024: 0x1297 ---- 0b1001010010111  
0x80000028: 0x8c2a823 ---- 0b100011000010101010000010001  
0x8000002c: 0xfd5ff06f ---- 0b11111101010111111111000001101111  
0x80000030: 0x97 ---- 0b10010111  
0x80000034: 0x58080e7 ---- 0b101100000001000000011100111  
0x80000038: 0x97 ---- 0b10010111
```



## ■ 执行指令execute

- 执行指令

```
def execute(cpu, mem, inst):  
    op = get_opcode(inst)  
    proc = get_exec_proc(op)  
  
    if proc == None:  
        set_trap(cpu, ILLEGAL_INSTRUCTION, inst)  
    else:  
        proc(cpu, mem, inst)  
  
    #指令执行中发生了异常  
    if is_trap(cpu):  
        pass  
    else:  
        clear_trap(cpu)
```



## ■ 执行指令execute

- 辅助函数：指令解析

- `get_opcode`

```
def get_opcode(i):  
    return i & 0x7f
```

- `get_rd`

- `get_funct3`

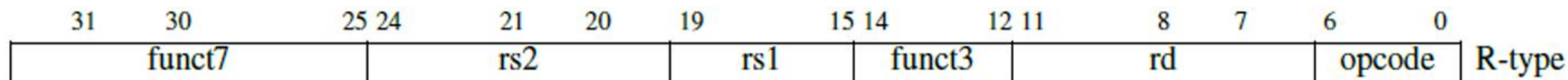
- `get_rs1`

- `get_rs2`

- `get_funct7`

- ... ..

```
def get_rd(i):  
    return (i >> 7) & 0x1f
```



## ■ 执行指令execute

- 辅助函数：指令分发get\_exec\_proc

```
def get_exec_proc(op):  
    if op in opcodes:  
        return opcodes[op]  
    else:  
        return None
```

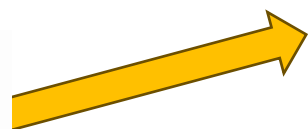


```
opcodes = {  
    LUI    : ex_lui,  
    AUIPC  : ex_auipc,  
    JAL    : ex_jal,  
    JALR   : ex_jalr,  
    B_TYPE: ex_branch,  
    LOAD   : ex_load,  
    S_TYPE: ex_store,  
    I_TYPE: ex_itype,  
    R_TYPE: ex_rtype,  
    FENCE  : ex_fence,  
    CSR    : ex_csr,  
}
```

## ■ LUI指令

• `ex_lui`

```
def ex_lui(cpu, mem, inst):  
    rd, imm = decode_u_type(inst)  
  
    set_reg(cpu, rd, imm)  
  
    pc_inc4(cpu)
```



```
def decode_u_type(inst):  
    rd = get_rd(inst)  
    imm = get_u_imm(inst)  
  
    return rd, imm
```

```
def get_u_imm(i):  
    return i & 0xffff_f000
```

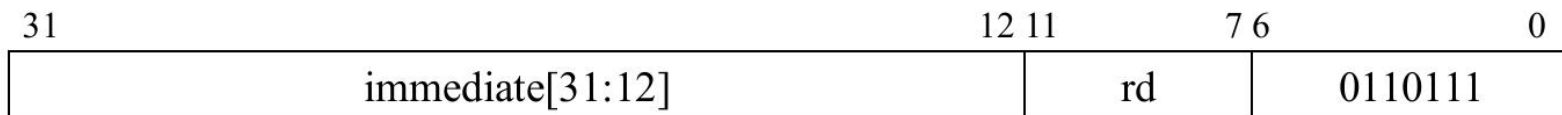
**lui**    rd, immediate

$x[rd] = \text{sext}(\text{immediate}[31:12] \ll 12)$

高位立即数加载 (*Load Upper Immediate*). U-type, RV32I and RV64I.

将符号位扩展的 20 位立即数 *immediate* 左移 12 位，并将低 12 位置零，写入  $x[rd]$  中。

压缩形式: **c.lui** rd, imm



## ■ AUIPC指令

- ex\_auipc

```
def ex_auipc(cpu, mem, inst):  
    rd, imm = decode_u_type(inst)  
    set_reg(cpu, rd, imm + get_pc(cpu))  
  
    pc_inc4(cpu)
```

```
def set_reg(cpu, r, val):  
    if r == 'zero' or r == 'x0' or r == 0:  
        return  
  
    if type(r) == str:  
        if r in REG_NAME:  
            r = REG_NAME[r]  
        else:  
            err('reg', f'非法寄存器{r}')  
    cpu['regs'][r] = val
```

```
def pc_inc4(cpu):  
    cpu['pc'] = cpu['pc'] + 4
```

**auipc** rd, immediate

$x[rd] = pc + sext(immediate[31:12] \ll 12)$

PC 加立即数 (Add Upper Immediate to PC). U-type, RV32I and RV64I.

把符号位扩展的 20 位（左移 12 位）立即数加到 *pc* 上，结果写入 *x[rd]*。

|                  |       |     |         |
|------------------|-------|-----|---------|
| 31               | 12 11 | 7 6 | 0       |
| immediate[31:12] |       | rd  | 0010111 |

## ■ JAL指令

- ex\_jal

```
def ex_jal(cpu, mem, inst):  
    rd, imm = decode_jal(inst)  
  
    pc = get_pc(cpu)  
  
    set_reg(cpu, rd, pc + 4)  
    set_pc(cpu, pc + imm)
```



```
def decode_jal(inst):  
    return get_rd(inst), get_j_imm(inst)
```

```
def get_j_imm(i):  
    v = ((i & 0x8000_0000) >> 11) | ((i & 0x7fe0_0000) >> 20) | \  
        ((i & 0x0010_0000) >> 9) | (i & 0x000ff000)  
    return sign_extend(v, 21, 32)
```

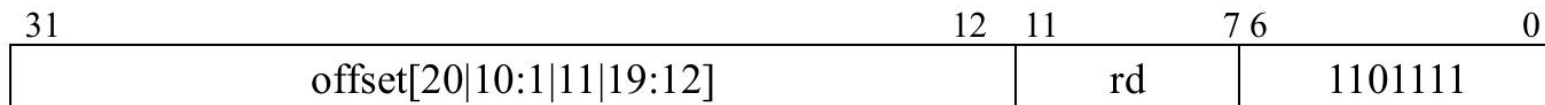
**jal** rd, offset

跳转并链接 (*Jump and Link*). J-type, RV32I and RV64I.

把下一条指令的地址( $pc+4$ ), 然后把 $pc$  设置为当前值加上符号位扩展的  $offset$ .  $rd$  默认为  $x1$ 。

压缩形式: **c.j** offset; **c.jal** offset

$x[rd] = pc+4; pc += sext(offset)$



## ■ JALR指令

- ex\_jalr

```
def ex_jalr(cpu, mem, inst):  
    _, rd, rs1, imm = decode_i_type(inst)  
  
    r1 = get_reg(cpu, rs1)  
  
    pc = get_pc(cpu)  
  
    set_reg(cpu, rd, pc+4)  
    set_pc(cpu, (r1 + imm) & 0xffff_fffe)
```

**jalr** rd, offset(rs1) t = pc+4; pc=(x[rs1]+sext(offset))&~1; x[rd]=t

跳转并寄存器链接 (*Jump and Link Register*). I-type, RV32I and RV64I.

把 *pc* 设置为  $x[rs1] + \text{sign-extend}(\text{offset})$ , 把 计算出的地址的最低有效位设为 0, 并将原 *pc+4* 的值写入  $f[rd]$ 。 *rd* 默认为 x1。

压缩形式: **c.jr** rs1; **c.jalr** rs1

|              |       |       |       |     |         |
|--------------|-------|-------|-------|-----|---------|
| 31           | 20 19 | 15 14 | 12 11 | 7 6 | 0       |
| offset[11:0] |       | rs1   | 010   | rd  | 1100111 |



## ■ JALR指令

- ex\_jalr

```
def ex_jalr(cpu, mem, inst):  
    _, rd, rs1, imm = decode_i_type(inst)  
  
    r1 = get_reg(cpu, rs1)  
  
    pc = get_pc(cpu)  
  
    set_reg(cpu, rd, pc+4)  
    set_pc(cpu, (r1 + imm) & 0xffff_fffe)
```

**jalr** rd, offset(rs1) t = pc+4; pc=(x[rs1]+sext(offset))&~1; x[rd]=t

跳转并寄存器链接 (*Jump and Link Register*). I-type, RV32I and RV64I.

把 *pc* 设置为  $x[rs1] + \text{sign-extend}(\text{offset})$ , 把 计算出的地址的最低有效位设为 0, 并将原 *pc*+4 的值写入  $f[rd]$ 。 *rd* 默认为 x1。

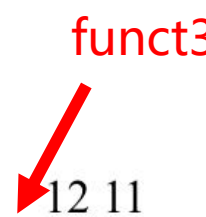
压缩形式: **c.jr** rs1; **c.jalr** rs1

|              |       |       |       |     |         |
|--------------|-------|-------|-------|-----|---------|
| 31           | 20 19 | 15 14 | 12 11 | 7 6 | 0       |
| offset[11:0] |       | rs1   | 010   | rd  | 1100111 |



## ■ branch指令

- beq、bne、blt、bge、bltu、bgeu



| 31           | 25 24 | 20 19 | 15 14 | 12 11       | 7 6     | 0 |        |
|--------------|-------|-------|-------|-------------|---------|---|--------|
| imm[12 10:5] | rs2   | rs1   | 000   | imm[4:1 11] | 1100011 |   | B beq  |
| imm[12 10:5] | rs2   | rs1   | 001   | imm[4:1 11] | 1100011 |   | B bne  |
| imm[12 10:5] | rs2   | rs1   | 100   | imm[4:1 11] | 1100011 |   | B blt  |
| imm[12 10:5] | rs2   | rs1   | 101   | imm[4:1 11] | 1100011 |   | B bge  |
| imm[12 10:5] | rs2   | rs1   | 110   | imm[4:1 11] | 1100011 |   | B bltu |
| imm[12 10:5] | rs2   | rs1   | 111   | imm[4:1 11] | 1100011 |   | B bgeu |

**beq** rs1, rs2, offset

if (rs1 == rs2) pc += sext(offset)

相等时分支 (*Branch if Equal*). B-type, RV32I and RV64I.

若寄存器 x[rs1]和寄存器 x[rs2]的值相等,把 pc 的值设为当前值加上符号位扩展的偏移 offset.

压缩形式: **c.beqz** rs1, offset

| 31              | 25 24 | 20 19 | 15 14 | 12 11          | 7 6     | 0 |
|-----------------|-------|-------|-------|----------------|---------|---|
| offset[12 10:5] | rs2   | rs1   | 000   | offset[4:1 11] | 1100011 |   |

## ■ branch指令

```
def ex_branch(cpu, mem, inst):  
    funct3, rs1, rs2, imm = decode_b_type(inst)
```

```
    r1 = get_reg(cpu, rs1)  
    r2 = get_reg(cpu, rs2)
```

```
    r1_u = r1 & 0xffff_ffff  
    r2_u = r2 & 0xffff_ffff
```

```
    target = get_b_imm(inst) + get_pc(cpu)
```

```
    if funct3 not in [BEQ, BNE, BLT, BGE, BLTU, BGEU]:  
        set_trap(cpu, ILLEGAL_INSTRUCTION, inst)  
        return
```

```
    if (funct3 == BEQ and r1 == r2) or (funct3 == BNE and r1 != r2) or \  
        (funct3 == BLT and r1 < r2) or (funct3 == BGE and r1 >= r2) or \  
        (funct3 == BLTU and r1_u < r2_u) or (funct3 == BGEU and r1_u >= r2_u):  
        set_pc(cpu, target)
```

```
    else:  
        pc_inc4(cpu)
```

```
def decode_b_type(inst):  
    return get_funct3(inst), get_rs1(inst), get_rs2(inst), get_b_imm(inst)
```

非法指令

转移

## Itype指令

| 31        | 25 24 | 20 19 | 15 14 | 12 11 | 7 6     | 0 |         |
|-----------|-------|-------|-------|-------|---------|---|---------|
| imm[11:0] |       | rs1   | 000   | rd    | 0010011 |   | I addi  |
| imm[11:0] |       | rs1   | 010   | rd    | 0010011 |   | I slti  |
| imm[11:0] |       | rs1   | 011   | rd    | 0010011 |   | I sltiu |
| imm[11:0] |       | rs1   | 100   | rd    | 0010011 |   | I xori  |
| imm[11:0] |       | rs1   | 110   | rd    | 0010011 |   | I ori   |
| imm[11:0] |       | rs1   | 111   | rd    | 0010011 |   | I andi  |
| 0000000   | shamt | rs1   | 001   | rd    | 0010011 |   | I slli  |
| 0000000   | shamt | rs1   | 101   | rd    | 0010011 |   | I srli  |
| 0100000   | shamt | rs1   | 101   | rd    | 0010011 |   | I srai  |

移位位数

**addi** rd, rs1, immediate

$x[rd] = x[rs1] + \text{sext}(\text{immediate})$

加立即数 (*Add Immediate*). I-type, RV32I and RV64I.

把符号位扩展的立即数加到寄存器  $x[rs1]$  上, 结果写入  $x[rd]$ 。忽略算术溢出。

压缩形式: **c.li** rd, imm; **c.addi** rd, imm; **c.addi16sp** imm; **c.addi4spn** rd, imm

| 31              | 20 19 | 15 14 | 12 11 | 7 6 | 0       |
|-----------------|-------|-------|-------|-----|---------|
| immediate[11:0] |       | rs1   | 000   | rd  | 0010011 |

## Itype指令

```
def ex_itype(cpu, mem, inst):  
    funct3, rd, rs1, imm = decode_i_type(inst)
```

```
    r1 = get_reg(cpu, rs1)  
    shamt = imm & 0x1f
```

```
    if funct3 == ADDI:  
        result = r1 + imm
```

```
    elif funct3 == SLTI:
```

```
        if r1 < imm:  
            result = 1
```

```
        else:
```

```
            result = 0
```

```
    elif funct3 == SLTIU:
```

```
        if (r1 & 0xffff ffff) < (imm & 0xffff ffff):  
            result = 1
```

```
        else:
```

```
            result = 0
```

```
    elif funct3 == XORI:
```

```
        result = r1 ^ imm
```

```
    elif funct3 == ORI:
```

```
        result = r1 | imm
```

```
    elif funct3 == ANDI:
```

```
        result = r1 & imm
```

算术右移

```
    elif funct3 == SLLI:
```

```
        result = r1 << shamt
```

```
    elif funct3 == SRI:
```

```
        if funct7 == SRLI:
```

```
            result = (r1 & 0xffff ffff) >> shamt
```

```
        elif funct7 == SRAI:
```

```
            result = r1 >> shamt
```

```
        else:
```

```
            set_trap(cpu, ILLEGAL_INST, inst)
```

```
            return
```

```
    else:
```

```
        set_trap(cpu, ILLEGAL_INST, inst)
```

```
        return
```

```
    set_reg(cpu, rd, result)
```

```
    pc_inc4(cpu)
```

逻辑右移

无符号数比较

## ■ Rtype指令

| 31      | 25 24 | 20 19 | 15 14 | 12 11 | 7 6     | 0 |       |
|---------|-------|-------|-------|-------|---------|---|-------|
| 0000000 | rs2   | rs1   | 000   | rd    | 0110011 |   | R add |
| 0100000 | rs2   | rs1   | 000   | rd    | 0110011 |   | R sub |
| 0000000 | rs2   | rs1   | 001   | rd    | 0110011 |   | R sll |
| 0000000 | rs2   | rs1   | 010   | rd    | 0110011 |   | R slt |
| 0000000 | rs2   | rs1   | 011   | rd    | 0110011 |   | Rsltu |
| 0000000 | rs2   | rs1   | 100   | rd    | 0110011 |   | R xor |
| 0000000 | rs2   | rs1   | 101   | rd    | 0110011 |   | R srl |
| 0100000 | rs2   | rs1   | 101   | rd    | 0110011 |   | R sra |
| 0000000 | rs2   | rs1   | 110   | rd    | 0110011 |   | R or  |
| 0000000 | rs2   | rs1   | 111   | rd    | 0110011 |   | R and |

**add** rd, rs1, rs2

$$x[rd] = x[rs1] + x[rs2]$$

加 (Add). R-type, RV32I and RV64I.

把寄存器 x[rs2]加到寄存器 x[rs1]上，结果写入 x[rd]。忽略算术溢出。

压缩形式: **c.add** rd, rs2; **c.mv** rd, rs2

| 31      | 25 24 | 20 19 | 15 14 | 12 11 | 7 6     | 0 |
|---------|-------|-------|-------|-------|---------|---|
| 0000000 | rs2   | rs1   | 000   | Rd    | 0110011 |   |



## ■ Rtype指令

```
def ex_rtype(cpu, mem, inst):
    funct3, funct7, rd, rs1, rs2 = decode_r_type(inst)

    r1 = get_reg(cpu, rs1)
    r2 = get_reg(cpu, rs2)

    if funct3 == ADDSUB:
        if funct7 == ADD:
            result = r1 + r2
        elif funct7 == SUB:
            result = r1 - r2
        else:
            set_trap(cpu, ILLEGAL_INST, inst)
            return
    elif funct3 == SLL:
        result = r1 << (r2 & 0x1f)
    elif funct3 == SLT:
        if r1 < r2:
            result = 1
        else:
            result = 0
    elif funct3 == SLTU:
        if (r1 & 0xffff_ffff) < (r2 & 0xffff_ffff):
            result = 1
        else:
            result = 0
```

无符号数比较

```
    elif funct3 == SR:
        if funct7 == SRL:
            result = (r1 & 0xffff_ffff) >> r2
        elif funct7 == SRA:
            result = r1 >> r2
        else:
            set_trap(cpu, ILLEGAL_INST, inst)
            return
    elif funct3 == XOR:
        result = r1 ^ imm
    elif funct3 == ORI:
        result = r1 | imm
    elif funct3 == ANDI:
        result = r1 & imm
    else:
        set_trap(cpu, ILLEGAL_INST, inst)
        return

    set_reg(cpu, rd, result)
    pc_inc4(cpu)
```



## ■ 访存指令: load

| 31        | 25 24 | 20 19 | 15 14 | 12 11 | 7 6     | 0 |       |
|-----------|-------|-------|-------|-------|---------|---|-------|
| imm[11:0] |       | rs1   | 000   | rd    | 0000011 |   | I lb  |
| imm[11:0] |       | rs1   | 001   | rd    | 0000011 |   | I lh  |
| imm[11:0] |       | rs1   | 010   | rd    | 0000011 |   | I lw  |
| imm[11:0] |       | rs1   | 100   | rd    | 0000011 |   | I lbu |
| imm[11:0] |       | rs1   | 101   | rd    | 0000011 |   | I lhu |

funct3

} 有符号

} 无符号

**lb** rd, offset(rs1)  $x[rd] = \text{sext}(M[x[rs1] + \text{sext}(\text{offset})][7:0])$

字节加载 (Load Byte). I-type, RV32I and RV64I.

从地址  $x[rs1] + \text{sign-extend}(\text{offset})$  读取一个字节，经符号位扩展后写入  $x[rd]$ 。

| 31           | 20 19 | 15 14 | 12 11 | 7 6 | 0       |
|--------------|-------|-------|-------|-----|---------|
| offset[11:0] |       | rs1   | 000   | rd  | 0000011 |

**lbu** rd, offset(rs1)  $x[rd] = M[x[rs1] + \text{sext}(\text{offset})][7:0]$

无符号字节加载 (Load Byte, Unsigned). I-type, RV32I and RV64I.

从地址  $x[rs1] + \text{sign-extend}(\text{offset})$  读取一个字节，经零扩展后写入  $x[rd]$ 。

| 31           | 20 19 | 15 14 | 12 11 | 7 6 | 0       |
|--------------|-------|-------|-------|-----|---------|
| offset[11:0] |       | rs1   | 100   | rd  | 0000011 |

## ■ load指令

```
def ex_load(cpu, mem, inst):
    funct3, rd, rs1, imm = decode_i_type(inst)

    r1 = get_reg(cpu, rs1)

    addr = r1 + imm

    unsigned = -1

    if funct3 == LB:
        size = 1
    elif funct3 == LH:
        size = 2
    elif funct3 == LW:
        size = 4
    elif funct3 == LBU:
        size = 1
        unsigned = 0xff
    elif funct3 == LHU:
        size = 2
        unsigned = 0xffff
    else:
        set_trap(cpu, ILLEGAL_INST, inst)
        return
```

IO设备



```
if check_io_addr(addr):      # io设备
    val = io_load(addr, size)
elif check_addr(mem, addr): # mem
    val = mem_load(mem, addr, size)
else:
    set_trap(cpu, LOAD_ACCESS_FAULT, addr) #读地址异常
    return

if unsigned != -1:
    val = sign_extend(val, size, 32) #有符号数读: lb、lh、lw
else:
    val = val & unsigned             #无符号数读: lbu、lhu

set_reg(cpu, rd, val)
pc_inc4(cpu)
```

## ■ 访存指令：store

|           |       |       |       |       |          |         |      |
|-----------|-------|-------|-------|-------|----------|---------|------|
| 31        | 25 24 | 20 19 | 15 14 | 12 11 | 7 6      | 0       |      |
| imm[11:5] |       | rs2   | rs1   | 000   | imm[4:0] | 0100011 | S sb |
| imm[11:5] |       | rs2   | rs1   | 001   | imm[4:0] | 0100011 | S sh |
| imm[11:5] |       | rs2   | rs1   | 010   | imm[4:0] | 0100011 | S sw |

funct3

**sb** rs2, offset(rs1)

$$M[x[rs1] + sext(offset) = x[rs2][7:0]$$

存字节 (*Store Byte*). S-type, RV32I and RV64I.

将  $x[rs2]$  的低位字节存入内存地址  $x[rs1] + sign-extend(offset)$ 。

|              |       |       |       |       |             |         |
|--------------|-------|-------|-------|-------|-------------|---------|
| 31           | 25 24 | 20 19 | 15 14 | 12 11 | 7 6         | 0       |
| offset[11:5] |       | rs2   | rs1   | 000   | offset[4:0] | 0100011 |

## ■ store指令

```
def ex_store(cpu, mem, inst):  
    funct3, rs1, rs2, imm = decode_store(inst)
```

```
    r1 = get_reg(cpu, rs1)  
    r2 = get_reg(cpu, rs2)
```

```
    addr = r1 + imm
```

```
    if funct3 == SB:  
        size = 1  
        r2 = r2 & 0xff  
    elif funct3 == SH:  
        size = 2  
        r2 = r2 & 0xffff  
    elif funct3 == SW:  
        size = 4
```

```
    else:  
        set_trap(cpu, ILLEGAL_INST, inst)  
        return
```

```
    if check_io_addr(addr):  
        io_store(addr, size, r2)  
    elif check_addr(mem, addr):  
        mem_store(mem, addr, size, r2)
```

```
    else:  
        set_trap(cpu, STORE_ACCESS_FAULT, addr) #写地址异常  
        return
```

```
def decode_store(inst):  
    return get_funct3(inst), get_rs1(inst), get_rs2(inst), get_s_imm(inst)
```

IO设备

## ■ 访问I/O设备

- 外设通常通过内存映射或I/O端口映射到CPU的地址空间。
- 内存映射的外设可以直接通过读写内存地址来访问，而I/O端口映射的外设则需要通过特定的I/O指令来访问。
- **内存映射I/O (MMIO)**：CPU通过读写内存地址来与外设通信。这些地址被映射到外设的寄存器，从而实现对设备的控制和数据交换。
- **端口映射I/O (PIO)**：CPU使用特定的输入/输出指令（如IN和OUT指令）来访问特定的I/O端口地址，这些地址对应于外设的寄存器。
- CPU与外设通讯方式
  - 轮询
  - 中断
  - DMA

## ■ I/O设备：16550 UART

- 16550 UART是一款通用异步收发传输器（UART），它包含多个寄存器用于控制和监视串口通信。这些寄存器共同工作，使得16550 UART能够处理串行通信的各种需求，包括数据传输、中断控制、FIFO管理、调制解调器控制等。

| LCR(7) | Address Offset | Register Name | Access Type | Description                                     |
|--------|----------------|---------------|-------------|-------------------------------------------------|
| 0      | 0x1000         | RBR           | RO          | Receiver Buffer Register                        |
| 0      | 0x1000         | THR           | WO          | Transmitter Holding Register                    |
| 0      | 0x1004         | IER           | R/W         | Interrupt Enable Register                       |
| x      | 0x1008         | IIR           | RO          | Interrupt Identification Register               |
| x      | 0x1008         | FCR           | WO          | FIFO Control Register                           |
| 1      | 0x1008         | FCR           | RO          | FIFO Control Register                           |
| x      | 0x100C         | LCR           | R/W         | Line Control Register                           |
| x      | 0x1010         | MCR           | R/W         | Modem Control Register                          |
| x      | 0x1014         | LSR           | R/W         | Line Status Register                            |
| x      | 0x1018         | MSR           | R/W         | Modem Status Register                           |
| x      | 0x101C         | SCR           | R/W         | Scratch Register                                |
| 1      | 0x1000         | DLL           | R/W         | Divisor Latch (Least Significant Byte) Register |
| 1      | 0x1004         | DLM           | R/W         | Divisor Latch (Most Significant Byte) Register  |



## ■ I/O设备：16550 UART模拟

- uart地址映射

- uart\_base = 0x1000\_0000

- 读串口:

```
def io_load(addr, size):  
    if addr == uart_base + 0:  
        i = input()  
        return int(i)
```

```
    return 0
```

- 写串口

```
def io_store(addr, size, val):  
    if addr == uart_base + 0:  
        print(hex(val)[2:],end='')
```

## ■ I/O设备：16550 UART模拟

- 汇编程序访问串口
  - putch: 向串口写一个字符
  - getch: 从串口读一个字符

putch:

```
    addi sp, sp, -16
    sw ra, 0(sp)
    sw s0, 4(sp)

    li s0, 0x10000000 #uart base
    sb a0, 0(s0)

    lw ra, 0(sp)
    lw s0, 4(sp)

    addi sp, sp, 16
    ret
```

getch:

```
    addi sp, sp, -16
    sw ra, 0(sp)
    sw s0, 4(sp)

    li s0, 0x10000000
    lb a0, 0(s0)

    lw ra, 0(sp)
    lw s0, 4(sp)

    addi sp, sp, 16
    ret
```

## ■ 系统指令：CSR读写、ecall、ebreak

- CSR：控制状态寄存器，空间大小为 $2^{12}$

| 31           | 25 24 | 20 19 | 15 14 | 12 11 | 7 6     | 0 |          |
|--------------|-------|-------|-------|-------|---------|---|----------|
| 000000000000 |       | 00000 | 000   | 00000 | 1110011 |   | I ecall  |
| 000000000001 |       | 00000 | 000   | 00000 | 1110011 |   | I ebreak |
| csr          |       | rs1   | 001   | rd    | 1110011 |   | I csrrw  |
| csr          |       | rs1   | 010   | rd    | 1110011 |   | I csrrs  |
| csr          |       | rs1   | 011   | rd    | 1110011 |   | I csrrc  |
| csr          |       | zimm  | 101   | rd    | 1110011 |   | I csrrwi |
| csr          |       | zimm  | 110   | rd    | 1110011 |   | I csrrsi |
| csr          |       | zimm  | 111   | rd    | 1110011 |   | I csrrci |

|         |       |       |     |       |         |              |
|---------|-------|-------|-----|-------|---------|--------------|
| 0001000 | 00010 | 00000 | 000 | 00000 | 1110011 | R sret       |
| 0011000 | 00010 | 00000 | 000 | 00000 | 1110011 | R mret       |
| 0001000 | 00101 | 00000 | 000 | 00000 | 1110011 | R wfi        |
| 0001001 | rs2   | rs1   | 000 | 00000 | 1110011 | R sfence.vma |

特权指令



## ■ 系统指令：CSR读写、ecall、ebreak

- ex csr

```
def ex_csr(cpu, mem, inst):
    funct3, rd, rs1, csr_no = decode_i_type(inst)

    if funct3 == 0: # ecall、ebreak、mret、sret、wfi
        if csr_no == EBREAK:
            return ex_ebreak(cpu, mem, inst)
        elif csr_no == ECALL:
            return ex_ecall(cpu, mem, inst)
        elif csr_no == MRET:
            return ex_mret(cpu, mem, inst)
        elif csr_no == SRET:
            return ex_sret(cpu, mem, inst)
        elif csr_no == WFI:
            return ex_wfi(cpu, mem, inst)
        else:
            set_trap(cpu, ILLEGAL_INST, inst)
            return
```

```
old_val = get_csr(cpu, csr_no)

if funct3 == CSRRW:
    new_val = r1
elif funct3 == CSRRS:
    new_val = old_val | r1
elif funct3 == CSRRC:
    new_val = old_val & (~r1)
elif funct3 == CSRRWI:
    new_val = rs1
elif funct3 == CSRRSI:
    new_val = old_val | rs1
elif funct3 == CSRRCI:
    new_val = old_val & ((~rs1) & 0x1f)
else:
    set_trap(cpu, ILLEGAL_INST, inst)
    return

set_reg(cpu, rd, old_val)
set_csr(cpu, csr_no, new_val)

pc_inc4(cpu)
```

## ■ RISC-V特权模式

- 在CPU中增加特权模式mode, 初始时为机器模式3

```
cpu = {  
    'pc': 0,  
    'regs': [0 for _ in range(32)],  
    'csr': [0 for _ in range(4096)],  
  
    'trap': -1,  
    'trap_val': 0,  
  
    'mode': 3, # 0: user, 1: supervisor, 3: machine  
}
```

```
def get_mode(cpu):  
    return cpu['mode']  
  
def set_mode(cpu, mode):  
    old_mode = get_mode(cpu)  
    cpu['mode'] = mode  
  
    return old_mode
```

| Encoding | Name       | Abbreviation |
|----------|------------|--------------|
| 00       | User       | U            |
| 01       | Supervisor | S            |
| 11       | Machine    | M            |



## ■ RISC-V异常和中断

- 中断和异常号

- 模拟器不处理中断

#interrupt and exception

```
ILLEGAL_INST = 2
BREAKPOINT = 3
LOAD_ADDRESS_MISALIGNED = 4
LOAD_ACCESS_FAULT = 5
STORE_ADDRESS_MISALIGNED = 6
STORE_ACCESS_FAULT = 7
ECALL_FROM_UMODE = 8
ECALL_FROM_SMODE = 9
ECALL_FROM_MMODE = 11
INST_PAGE_FAULT = 12
LOAD_PAGE_FAULT = 13
STORE_PAGE_FAULT = 15
```

| Interrupt / <del>Exception</del><br>mcause[XLEN-1] | Exception Code<br>mcause[XLEN-2:0] | Description                    |
|----------------------------------------------------|------------------------------------|--------------------------------|
| 1                                                  | 1                                  | Supervisor software interrupt  |
| 1                                                  | 3                                  | Machine software interrupt     |
| 1                                                  | 5                                  | Supervisor timer interrupt     |
| 1                                                  | 7                                  | Machine timer interrupt        |
| 1                                                  | 9                                  | Supervisor external interrupt  |
| 1                                                  | 11                                 | Machine external interrupt     |
| 0                                                  | 0                                  | Instruction address misaligned |
| 0                                                  | 1                                  | Instruction access fault       |
| 0                                                  | 2                                  | Illegal instruction            |
| 0                                                  | 3                                  | Breakpoint                     |
| 0                                                  | 4                                  | Load address misaligned        |
| 0                                                  | 5                                  | Load access fault              |
| 0                                                  | 6                                  | Store address misaligned       |
| 0                                                  | 7                                  | Store access fault             |
| 0                                                  | 8                                  | Environment call from U-mode   |
| 0                                                  | 9                                  | Environment call from S-mode   |
| 0                                                  | 11                                 | Environment call from M-mode   |
| 0                                                  | 12                                 | Instruction page fault         |
| 0                                                  | 13                                 | Load page fault                |
| 0                                                  | 15                                 | Store page fault               |

中断

异常



## ■ ebreak

- ex\_ebreak

```
def ex_ebreak(cpu, mem, inst):  
    set_trap(cpu, BREAKPOINT)
```

## ■ ecall

- ex\_ecall

```
def ex_ecall(cpu, mem, inst):  
    mode = get_mode(cpu)  
  
    if mode == 0:  
        set_trap(cpu, ECALL_FROM_UMODE)  
    elif mode == 1:  
        set_trap(cpu, ECALL_FROM_SMODE)  
    else mode == 3:  
        set_trap(cpu, ECALL_FROM_MMODE)  
  
    pc_inc4(cpu)
```

## ■ 特权指令：wfi

- 等待中断：ex\_wfi

```
def ex_wfi(cpu, mem, inst):  
    pc_inc4(cpu)
```

## ■ 特权指令：mret

- ex\_mret

```
def ex_mret(cpu, mem, inst):  
    mstatus = get_csr(cpu, 'mstatus')  
    current_mode = get_mode(cpu)  
  
    if current_mode != 3: #当前模式不为机器模式  
        err(f'在非机器模式[{current_mode}]下执行mret指令')
```

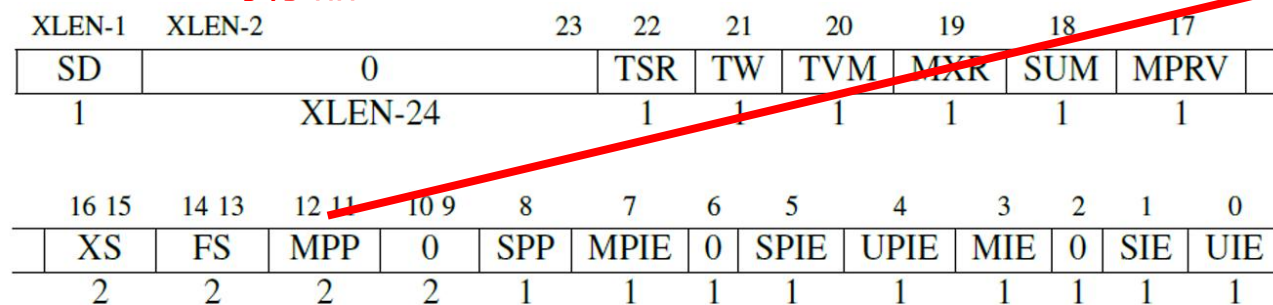
```
    prev_mode = get_mpp(mstatus)
```

```
    set_mode(cpu, prev_mode)
```

```
    set_pc(cpu, get_csr('mepc'))
```

```
def get_mpp(mstatus):  
    return (mstatus >> 10) & 3
```

mstatus寄存器





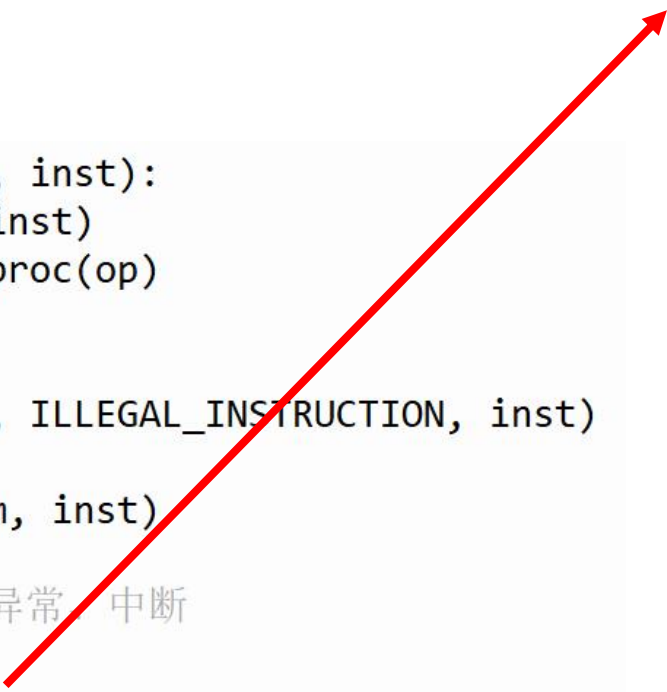
## ■ 处理异常

- ex\_trap

```
def execute(cpu, mem, inst):
    op = get_opcode(inst)
    proc = get_exec_proc(op)

    if proc == None:
        set_trap(cpu, ILLEGAL_INSTRUCTION, inst)
    else:
        proc(cpu, mem, inst)

    #指令执行中发生了异常 中断
    if is_trap(cpu):
        ex_trap(cpu)
```



```
def ex_trap(cpu):
    trap = get_trap(cpu)

    #设置异常原因
    set_csr(cpu, 'mcause', trap)

    #设置mtval
    if trap & 0x8000_0000: #中断
        set_csr(cpu, 'mtval', 0)
    else: #异常
        set_csr(cpu, 'mtval', get_trap_val(cpu))

    #设置异常返回地址
    set_csr(cpu, 'mepc', pc)

    mode = get_mode(cpu)
    mstatus = get_csr(cpu, 'mstatus')

    #保存mstatus中断标志IE位至MPIE, 保存当前模式至MPP
    mstatus = (mstatus & 0x8) << 4 | (mode & 0x3) << 11
    set_csr(cpu, 'mstatus', mstatus)

    #设置pc为中断异常处理入口地址
    set_pc(cpu, get_csr(cpu, 'mtvec'))

    #设置为机器模式
    set_mode(cpu, 3)

    #清除异常标志
    clear_trap(cpu)
```



## ■ 示例

```
.global _start

.data
x: .word 100
y: .word 200
z: .word 0
stack_end: .skip 1024
stack_start:

.text
_start:

    la sp, stack_start

    lw a0, x
    lw a1, y
    add a2, a1, a0

    sw a2, z, t0

    j _start
```

编译: riscv-none-elf-gcc -Ttext=0x8000000 -mno-relax -  
march=rv32i\_zicsr -nostdlib -ffreestanding -o test test0.s  
生成镜像: riscv-none-elf-objcopy -O binary test.bin test

### Disassembly of section .text:

```
08000000 <_start>:
80000000: 00001117      auipc    sp,0x1
80000004: 4b010113      addi     sp,sp,1200 # 80014b0 <__BSS_END__>
80000008: 00001517      auipc    a0,0x1
8000000c: 09c52503      lw       a0,156(a0) # 80010a4 <__DATA_BEGIN__>
80000010: 00001597      auipc    a1,0x1
80000014: 0985a583      lw       a1,152(a1) # 80010a8 <y>
80000018: 00a58633      add      a2,a1,a0
8000001c: 00001297      auipc    t0,0x1
80000020: 08c2a823      sw       a2,144(t0) # 80010ac <z>
80000024: fddff06f      j        80000000 <_start>
```

## ■ 示例

```
>>> load_image(mem, 'test.bin', 0x8000_0000)
>>> set_pc(cpu, 0x8000_0000)
```

Disassembly of section .text:

```
08000000 <_start>:
8000000: 00001117      auipc    sp,0x1
8000004: 4b010113      addi     sp,sp,1200 # 80014b0 <__BSS_END__>
8000008: 00001517      auipc    a0,0x1
800000c: 09c52503      lw       a0,156(a0) # 80010a4 <__DATA_BEGIN__>
8000010: 00001597      auipc    a1,0x1
8000014: 0985a583      lw       a1,152(a1) # 80010a8 <y>
8000018: 00a58633      add      a2,a1,a0
800001c: 00001297      auipc    t0,0x1
8000020: 08c2a823      sw       a2,144(t0) # 80010ac <z>
8000024: fddff06f      j        8000000 <_start>
```

```
>>> step(cpu, mem)
```

```
0x80000000: 0x1117      ---- 0b1000100010111
```

```
>>> step(cpu, mem)
```

```
0x80000004: 0x4b010113 ---- 0b1001011000000010000000100010011
```

```
>>> step(cpu, mem)
```

```
0x80000008: 0x1517      ---- 0b1010100010111
```

```
>>> hex(get_reg(cpu, 'sp'))
'0x800014b0'
```

```
>>> hex(get_reg(cpu, 'a0'))
'0x80001008'
```



## RV32I模拟器



- 模拟器结构
- LUI、AUIPC指令
- JAL、JALR、Branch指令
- 算术逻辑运算指令
- Load、store指令及外设模拟
- CSR及特权指令